

NAME: MAYANK MEHTA

USN: 1BF24CS171

A* ALGORITHM:

CODE:

```
import heapq
import time

GOAL = (1, 2, 3, 8, 0, 4, 7, 6, 5)

def heuristic(state):
    """Misplaced tiles: count of tiles not in their goal position."""
    return sum(1 for i, tile in enumerate(state) if tile != 0 and tile != GOAL[i])

def get_neighbors(state):
    blank = state.index(0)
    r, c = divmod(blank, 3)
    for dr, dc, move_name in [(-1, 0, 'Up'), (1, 0, 'Down'), (0, -1, 'Left'), (0, 1, 'Right')]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            s = list(state)
            s[blank], s[nr * 3 + nc] = s[nr * 3 + nc], s[blank]
            yield tuple(s)

def a_star_8puzzle(initial):
    open_set = [(heuristic(initial), 0, initial)]
    g_score = {initial: 0}
    came_from = {}
    closed = set()
    states_explored = 0

    while open_set:
        f, g, current = heapq.heappop(open_set)

        if current == GOAL:
            path = [current]
            while current in came_from:
                current = came_from[current]
```

```

        path.append(current)
        return path[::-1], states_explored

    if current in closed:
        continue
    closed.add(current)
    states_explored += 1

    for neighbor in get_neighbors(current):
        new_g = g + 1
        if neighbor not in g_score or new_g < g_score[neighbor]:
            g_score[neighbor] = new_g
            came_from[neighbor] = current
            heapq.heappush(open_set, (new_g + heuristic(neighbor),
new_g, neighbor))

    return None, states_explored

def get_move_direction(prev_state, next_state):
    prev_blank = prev_state.index(0)
    next_blank = next_state.index(0)
    prev_r, prev_c = divmod(prev_blank, 3)
    next_r, next_c = divmod(next_blank, 3)

    dr = next_r - prev_r
    dc = next_c - prev_c

    if dr == -1:
        return "Up"
    elif dr == 1:
        return "Down"
    elif dc == -1:
        return "Left"
    elif dc == 1:
        return "Right"
    return ""

def print_grid(state):
    for r in range(3):
        print(" ".join(str(x) for x in state[r*3:r*3+3]))

```

```

# --- Run ---
start = (2, 8, 3, 1, 6, 4, 7, 0, 5)

print("=== Initial State ===")
print_grid(start)
print("\n=== Final Goal State ===")
print_grid(GOAL)
print("\n=====\\n")

solution, states_explored = a_star_8puzzle(start)

if solution:
    depth = len(solution) - 1
    print(f"Solved in {depth} moves:\\n")
    for i, state in enumerate(solution):
        print_grid(state)
        if i < len(solution) - 1:
            move_dir = get_move_direction(state, solution[i+1])
            print(f"\\n      ↓ (Move blank {move_dir})\\n")

    b = 3.17
    time_space_complexity = b ** depth
    print("=====\\n")
    print(f"No. of Moves (Depth d): {depth}")
    print(f"No. of States Explored: {states_explored}")
    print(f"No. of States in Path: {len(solution)}")
    print(f"Complexity Analysis (b^d where b = {b}, d = {depth}):")
    print(f"Theoretical Time/Space Complexity (O(b^d)):")
    ~{time_space_complexity:.2f} nodes")
else:
    print("No solution exists.")

```

OUTPUT:

```
*** === Initial State ===
  2  8  3
  1  6  4
  7  0  5

=== Final Goal State ===
  1  2  3
  8  0  4
  7  6  5

=====

Solved in 5 moves:

  2  8  3
  1  6  4
  7  0  5

      ↓ (Move blank Up)

  2  8  3
  1  0  4
  7  6  5

      ↓ (Move blank Up)

  2  0  3
  1  8  4
  7  6  5

      ↓ (Move blank Left)

  0  2  3
  1  8  4
  7  6  5

      ↓ (Move blank Down)
```

```
1 2 3
0 8 4
7 6 5
```

↓ (Move blank Right)

```
1 2 3
8 0 4
7 6 5
```

=====

No. of Moves (Depth d): 5

No. of States Explored: 6

No. of States in Path: 6

Complexity Analysis (b^d where $b = 3.17$, $d = 5$):

Theoretical Time/Space Complexity ($O(b^d)$): ~320.11 nodes